

Revisão Sistemática da Adoção do Scrum no Desenvolvimento de Software

Diovana Cesconetto Vitorino¹, Mateus Coelho Santos¹, Romário Vitorino Ferreira^{2*}

¹Acadêmicos do curso de Sistemas de Informação do Afya Centro Universitário de Ji-Paraná, Ji-Paraná, Rondônia, Brasil.

²Especialista em Análise de Projeto de Sistemas, Docente do curso de Sistemas de Informação do Afya Centro Universitário de Ji-Paraná, Ji-Paraná, RO, Brasil.

*Autor(a) correspondente: Romário Vitorino Ferreira, Docente do curso de Sistemas de Informação do Afya Centro Universitário de Ji-Paraná. Av.Eng.Manfredo Barata Almeida da Fonseca, Jardim Aurélio Bernardi, Ji-Paraná – RO, CEP: 76907-524. E-mail: romario.ferreira@saolucasjiparana.edu.br

Editor: Prof. Dr. Jerônimo Vieira Dantas Filho

Recebido em: 14/09/2025 Aceito em: 16/10/2025 publicado em: 21/10/2025

Resumo

O desenvolvimento de software tem evoluído significativamente nas últimas décadas, impulsionado pela necessidade de entregas mais rápidas, flexíveis e centradas no cliente. Nesse contexto, as metodologias ágeis emergem como alternativas aos modelos tradicionais de desenvolvimento, oferecendo abordagens interativas e colaborativas. Este artigo tem como objetivo analisar, por meio de uma revisão da literatura, o uso da metodologia ágil Scrum como uma alternativa eficiente no desenvolvimento de software. Para isso, foram selecionadas obras de referência em uma biblioteca virtual, utilizando os termos “Engenharia de Software”, “Metodologias para Desenvolvimento de Software” e “Scrum”. A fundamentação teórica contempla a evolução da engenharia de software, os princípios do Manifesto Ágil e o detalhamento dos papéis, artefatos e cerimônias do framework Scrum. Os resultados apontam que o Scrum contribui para uma maior organização das equipes, entrega contínua de valor e melhor gestão de projetos em ambientes dinâmicos. Conclui-se que sua adoção depende da compreensão de seus fundamentos e do preparo das equipes para mudanças culturais e estruturais e que, quando bem implementado, o Scrum proporciona maior controle sobre prazos, qualidade da entrega e alinhamento com os objetivos do cliente, sendo uma alternativa viável e cada vez mais adotada pelas equipes de desenvolvimento de software.

Palavras-chave: Engenharia de Software. Metodologias Ágeis. Scrum.

Systematic Review of Scrum Adoption in Software Development

Abstract

Software development has evolved significantly in recent decades, driven by the need for faster, more flexible, and customer-centric delivery. In this context, methodologies have emerged as alternatives to traditional development models, offering interactive and collaborative approaches. This article aims to analyze, through a literature review, the use of the Agile Scrum methodology as an efficient alternative in software development. To this end, reference works were selected from a virtual library, using the terms "Software Engineering," "Software Development Methodologies," and "Scrum." The theoretical foundation encompasses the evolution of software engineering, the principles of the Agile Manifesto, and a detailed description of the Scrum framework's roles, artifacts, and ceremonies. The results indicate that Scrum contributes to greater team organization, continuous value delivery, and improved project management. It is concluded that its adoption depends on the understanding of its fundamentals and the preparation of teams for cultural and structural changes and that, when well implemented, Scrum provides greater control over deadlines, delivery quality and alignment with the client's objectives, being a viable alternative and increasingly applied by software development teams.

1. Introdução

A Engenharia de *Software* é o campo da computação responsável por aplicar métodos sistemáticos, disciplinados e mensuráveis ao desenvolvimento, operação e manutenção de software. Tradicionalmente, essa área adotou metodologias de desenvolvimento lineares e sequenciais, nas quais todas as etapas do projeto são rigidamente definidas desde o início, com pouca margem para mudanças. Embora essas abordagens tenham oferecido uma base sólida para a estruturação de projetos, suas limitações quanto à adaptação a requisitos voláteis e ao envolvimento contínuo do cliente tornaram-se evidentes em contextos mais dinâmicos e competitivos.

Nesse cenário, surgem as metodologias ágeis de desenvolvimento de *software*, que propõem uma ruptura com o modelo tradicional ao enfatizar flexibilidade, colaboração, entrega contínua de valor e resposta rápida às mudanças. Consolidado com a publicação do Manifesto Ágil em 2001, esse movimento destacou valores como indivíduos e interações mais que processos e ferramentas, e sistemas funcionando mais que documentação abrangente. Entre os métodos que se alinham a esses princípios, o Scrum tornou-se um dos *frameworks* mais adotados globalmente, especialmente por sua simplicidade e foco em entregas incrementais e iterativas.

O Scrum é estruturado em torno de papéis bem definidos, cada um com responsabilidades específicas voltadas à maximização do valor entregue ao cliente. O processo é guiado por cerimônias, que promovem comunicação contínua e ciclos de melhoria. Os artefatos, servem como instrumentos de transparência e controle do progresso do projeto. Além disso, o Scrum valoriza equipes auto-organizadas, multidisciplinares e comprometidas com a entrega de incrementos funcionais em curtos períodos, chamados de sprints.

Diante disso, este artigo tem como objetivo conceituar os métodos tradicionais e ágeis de desenvolvimento de *software* e apresentar o Scrum como uma alternativa viável e eficiente dentro do contexto do desenvolvimento ágil. Por meio de uma pesquisa bibliográfica baseada em

livros especializados e estudos recentes, pretende-se compreender os conceitos, práticas e fundamentos do Scrum.

1.2 Engenharia de Software

A Engenharia de *Software* é um ramo da Ciência da Computação que tem como objetivo estruturar o desenvolvimento dos sistemas com as melhores práticas, visando maior eficiência, manutenção e escalabilidade,

Genericamente podemos definir engenharia como a ciência de adquirir e de aplicar determinados conhecimentos abrangentes para realizar uma função ou objetivo. No caso da engenharia de software, observamos que é uma área de conhecimento da computação, cujos conhecimentos se relacionam basicamente com a especificação, o desenvolvimento e a manutenção de sistemas de software, com o objetivo de organizar o projeto e aumentar sua produtividade. Além destes objetivos, chamamos a atenção para outro objetivo importante, obtido pela aplicação de práticas de engenharia para o desenvolvimento de software: fornecer uma estrutura para a construção de software com alta qualidade (Sbrocco; Macedo, 2012, p. 36)

Pressman e Maxim apresentam que a este campo da computação, está fundamentado em camadas que precisam ser fundamentadas no comprometimento com a qualidade do *software*. Na imagem abaixo será possível observar todas as camadas, de acordo com os autores.



Fonte: Pressman e Maxim, 2021.

A evolução tecnológica das últimas décadas ocorreu fundamentada na engenharia de software, se antes, o desenvolvimento de sistemas ocorria sem padrão, estrutura planejamento, com o advento desta área da computação e sua popularização foi possível criar raízes sólidas para que cada vez mais fossem implementados com qualidade e robustez.

Software de computador continua a ser a tecnologia mais importante no cenário mundial. E é, também, um ótimo exemplo da lei das consequências não intencionais. Há 60 anos, ninguém poderia prever que o software se tornaria uma tecnologia indispensável para negócios, ciência e engenharia; que o software viabiliza a criação de novas tecnologias (p. ex., engenharia genética e nanotecnologia), a extensão de tecnologias existentes (p. ex., telecomunicações) e a mudança radical nas tecnologias mais antigas (p. ex., a mídia); que o software se tornaria a força motriz por trás da revolução do computador pessoal; que aplicativos de software seriam comprados pelos consumidores por meio de seus dispositivos móveis; que o software evoluiria lentamente de produto para serviço, à medida que empresas de software “sob demanda” oferecessem funcionalidade imediata (just-in-time) via um navegador Web; que uma empresa de software se tornaria maior e mais influente do que todas as empresas da era industrial; ou que uma vasta rede comandada por software evoluiria e modificaria tudo: de pesquisa em bibliotecas a compras feitas pelos consumidores, de discursos políticos a comportamentos de namoro entre jovens e (não tão jovens) adultos. (Pressman; Maxim, 2021, p.2)

Busca-se aproximar todas os envolvidos nos processos de desenvolvimento do sistema, seja da equipe de desenvolvimento do *software* ou do contratante, desta forma, o objetivo é entender as necessidades dos clientes e mapear os requisitos sistêmicos e definir em alto nível o que será implementado (Hirama, 2011).

1.3 Metodologias de Desenvolvimento Tradicionais

O foco deste artigo são as metodologias ágeis no processo de desenvolvimento, no entanto, será abordado brevemente as metodologias tradicionais de desenvolvimento, que foram as precursoras e que em ainda a depender do projeto, podem ser utilizadas. Abaixo serão apresentados os principais modelos tradicionais, de acordo com (Sbrocco; Macedo, 2012)

- Modelo sequencial linear: Algumas vezes chamado de ciclo de vida clássico ou modelo em cascata, o modelo sequencial linear sugere uma abordagem sistemática sequencial para o desenvolvimento de *software*, que começa no nível de sistema

e progride através da análise, projeto, codificação, teste e manutenção.

- Modelo de prototipagem: O cliente frequentemente define um conjunto de objetivos gerais para o *software*, mas não identifica detalhadamente requisitos de entrada, processamento ou saída. Em outros casos, o desenvolvedor pode estar inseguro da eficiência de um algoritmo, da adaptabilidade de um sistema operacional ou da forma que a interação homem/máquina deve assumir. Nessas e em muitas outras situações, um paradigma de prototipagem pode oferecer a melhor abordagem.
- Modelo Clássico: O modelo clássico foi o primeiro processo publicado de desenvolvimento de *software*, composto de um modelo de fácil entendimento, pois se baseia em uma sequência de etapas, e cada etapa tem associada ao seu término uma documentação padrão que deve ser aprovada para que se inicie a etapa imediatamente posterior. O modelo clássico tem sido um dos paradigmas mais amplamente utilizados na engenharia de *software*, contudo sua aplicabilidade tem sido questionada devido a alguns problemas que surgem quando utilizamos esse modelo.
- Desenvolvimento incremental: O desenvolvimento incremental combina elementos do modelo sequencial linear com elementos da filosofia interativa da prototipagem. A ideia básica de um modelo incremental é a aplicação de sequências lineares ao longo do tempo de desenvolvimento. Cada uma dessas sequências lineares deve produzir um incremento do projeto que está sendo desenvolvido, ou seja, dividimos todas as funcionalidades pretendidas e as agrupamos em sequências de atividades. Ao final de cada sequência linear, devemos sempre ter um produto pronto, que é parte integrante do projeto como um todo. Observe que durante o desenvolvimento de uma sequência linear

incorporamos as estratégias utilizadas pelo paradigma de prototipagem.

- Desenvolvimento em espiral: O software pode ser desenvolvido numa série de versões incrementais. Durante as primeiras iterações, pode-se produzir, por exemplo, protótipos, até que nas últimas iterações sejam versões cada vez mais completas do sistema. O modelo espiral é desenvolvido em atividades que formam uma estrutura, também chamada de regiões de tarefa.

1.4 Metodologias Ágeis

As metodologias de desenvolvimento ágeis são fundamentadas em princípios que buscam tornar o processo de gerenciamento de software mais simples, interativo e flexível, agregando valor para o cliente. Este tipo de desenvolvimento conta com equipe multidisciplinar, priorizando conversas em detrimento da documentação, sendo mais importante satisfazer a necessidade do cliente do que seguir um planejamento inicial (Maschietto; Rodrigues; Bianco, 2020).

Os conceitos das metodologias ágeis surgiram como uma forma de reação aos métodos tradicionais, que se caracterizam por serem muitas formações, documentações robustas, onde as equipes eram micro gerenciadas. De acordo com (Sbrocco; Macedo, 2012, p. 88)

Em 2001, uma importante reunião ocorrida em uma estação de esqui, nas montanhas nevadas do estado norte-americano de Utah, marcou definitivamente o surgimento e a propagação de paradigmas de desenvolvimento de software ágeis. Estavam presentes nessa reunião dezessete profissionais de software que já trabalhavam com os chamados métodos leves utilizados na época, a saber: Kent Beck, Mike Beedle, Arie Van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland e Dave Thomas. Estas pessoas representavam metodologias já utilizadas, como SCRUM, Extreme Programming, DSDM, Adaptive Software Development, Crystal, Feature Driven Development, entre outras.

Na reunião mencionada, os participantes perceberam que apesar de haver particularidades na forma de cada um conduzir o desenvolvimento do software, adaptando a sua necessidade, eles identificaram que existia alguns princípios que pareciam ser respeitados por todos quando os projetos eram bem-sucedidos, com base nisto, decidiram elaborar um documento contendo estes princípios. Um dos presentes era o inglês Martin Fowler que cunhou o termo “*agile*” para nomear o que estavam discutindo, desta forma, o documento por eles elaborado e assinado ficou chamado de “manifesto ágil”.

1.5 Manifesto Ágil

No início do século XXI, surgiram novas demandas para o desenvolvimento de *software*, em virtude do avanço dos *hardwares*, das linguagens de programação e dos ambientes cooperativos. Com a finalidade de substituir os métodos tradicionais de desenvolvimento, reduzir o tempo de entrega, garantir maior confiabilidade dos sistemas e agregar valor ao cliente, foi elaborado o Manifesto Ágil. Esse documento, produzido por especialistas da área, estabeleceu um conjunto de princípios e critérios destinados a orientar práticas de desenvolvimento de software mais eficientes, fundamentadas em equipes multidisciplinares, na participação ativa do cliente ao longo do processo e na priorização das funcionalidades em detrimento da documentação (Sbrocco; Macedo, 2012). A seguir, apresentam-se os doze princípios que compõem o Manifesto Ágil.

1. A prioridade é satisfazer ao cliente por meio de entregas contínuas e frequentes de *software* de valor.
2. Mudanças de requisitos são bem-vindas, mesmo em uma fase avançada do projeto. Processos ágeis esperam que a mudança traga uma vantagem competitiva ao cliente.
3. Entregas com frequência de *software* funcional, sempre na menor escala de tempo, de algumas semanas a alguns meses, preferindo sempre um período curto.

4. As equipes de negócio e de desenvolvimento devem trabalhar juntas diariamente durante o projeto.
5. Construa projetos objetivando manter uma equipe motivada, fornecendo ambiente, apoio e confiança necessários para realizar o trabalho.
6. A maneira mais eficiente de a informação circular entre a equipe de desenvolvimento é por uma conversa cara a cara.
7. Ter um *software* funcionando é a medida primária de progresso.
8. Processos ágeis promovem o desenvolvimento sustentável. Clientes, desenvolvedores e usuários devem ser capazes de manter um ritmo constante.
9. Atenção contínua à excelência técnica e a um bom projeto aumenta a agilidade.
10. Simplicidade é essencial.
11. As melhores soluções de arquitetura, requisitos e projetos nascem quando as equipes estão organizadas e trabalham de forma colaborativa.
12. Em intervalos regulares, a equipe deve refletir sobre como se tornar mais eficaz, então sintoniza e ajusta seu comportamento.

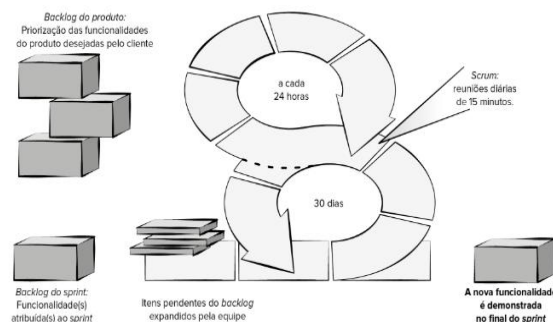
1.6 SCRUM

A metodologia de desenvolvimento Ágil Scrum segue os princípios do manifesto ágil, pois os seus criados Jeff Sutherland e Ken Schwaber foram dois dos seus signatários. Esta metodologia é baseada em seis características principais: flexibilidade dos resultados, flexibilidade dos prazos, times pequenos, revisões frequentes, colaboração e orientação a objetos (Sbrocco; Macedo, 2012).

O Scrum não é um processo padronizado onde segue-se metodicamente uma série de passos em sequência que garantidamente geram um produto de alta qualidade no prazo e dentro do orçamento e que deslumbre os clientes. Em vez disso, o Scrum é um framework para organizar e gerenciar o trabalho. O framework Scrum é baseado numa série de valores, princípios e práticas que fornecem a fundação onde sua organização vai adicionar sua própria implementação individual das práticas de engenharia relevantes e suas abordagens específicas para a realização das

práticas Scrum. O resultado vai ser uma versão do Scrum que é unicamente sua.

Dessa forma, o Scrum, com seus valores, práticas e princípios, constitui a base estrutural que consolida o desenvolvimento de *software*. Alterá-los implica comprometer o processo e pode acarretar o colapso do projeto. Contudo, essa metodologia permite certo grau de customização, mediante a incorporação de características específicas de cada organização. A figura a seguir apresenta o fluxo geral do Scrum, contemplando seus eventos e artefatos.



Fonte: Pressman e Maxim, 2021.

1.7 PAPEIS NO SCRUM

O desenvolvimento ágil com base nesta metodologia consiste em uma ou mais equipes Scrum, onde cada uma será composta de três papéis Scrum: *Product Owner*, *Scrum Master* e a Equipe de Desenvolvimento. Como esta metodologia é flexível e permite adaptações, em algumas organizações, pode haver mais papéis, no entanto, o que o Scrum oficialmente requer são estes três apresentados. (Rubin, 2017, p. 14).

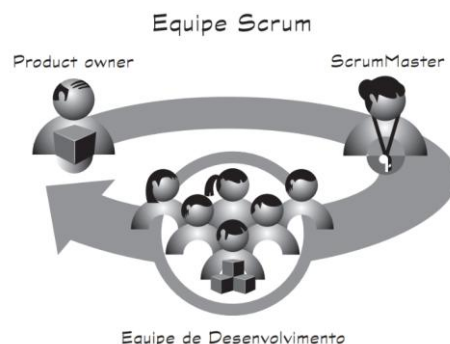
Nos tópicos abaixo serão apresentadas detalhadamente cada um destes papéis.

- **PO (Product Owner):** É o responsável pelo produto, tendo como principal função o gerenciamento da lista de funcionalidades ou requisitos do projeto que possuam valor comercial, vantagem competitiva ou relevância para a evolução sistêmica do cliente. Essa lista recebe o nome de *backlog*. Quando há necessidade de alteração na ordem de prioridade dos itens, a negociação deve ser realizada diretamente com o PO. Dessa forma, para que o Scrum seja executado com sucesso, é imprescindível que todos os membros da organização, independentemente do nível hierárquico,

respeitem a autoridade e a autonomia atribuídas ao *Product Owner* (Maschietto; Rodrigues; Bianco, 2020, p. 53).

- **Scrum Master:** O Scrum Master atua como um líder servidor para o Time Scrum, de forma a garantir que esta metodologia seja compreendida e aplicada por todos. Age como um facilitador, ajudando a equipe a resolver problemas e fazer melhorias no uso do Scrum. Também é responsável por proteger a equipe de interferências externas e exerce um papel de liderança na remoção de impedimentos que impactam a produtividade da equipe. É válido pontuar que o Scrum Master não tem autoridade para ter controle sobre a equipe, não sendo o mesmo papel tradicional do gerente de desenvolvimento, seu papel de fato é como um líder para em função da equipe. (Rubin, 2017, p. 16).
- **Equipe de Desenvolvimento:** A equipe de desenvolvimento é constituída por profissionais responsáveis pelo planejamento, implementação e teste do produto ao término de cada ciclo. Tal equipe organiza-se de maneira autônoma, definindo a melhor estratégia para atingir os objetivos estabelecidos pelo *Product Owner*. Recomenda-se que o grupo possua, coletivamente, todas as competências necessárias à produção de um software funcional e de elevada qualidade, sendo sua composição usualmente de cinco a nove integrantes (Maschietto, Rodrigues e Bianco, 2020, p. 55).

Na imagem apresentada abaixo é ilustrado todos os papéis integrantes da Equipe Scrum, pode-se observar que não há nível hierárquico entre eles.



Fonte: Rubin, 2017

1.8 Cerimônias do Scrum

O Scrum é estruturado em quatro cerimônias oficiais: **Planejamento da Sprint**, no qual são definidas e planejadas as metas de cada Sprint; **Reuniões Diárias**, encontros diários voltados ao autogerenciamento e à organização da equipe de desenvolvimento; **Revisão da Sprint**, momento em que o produto e o projeto são analisados e adaptados; e, por fim, a **Retrospectiva da Sprint**, na qual são discutidos os pontos fortes e fracos da equipe, identificando-se práticas a serem mantidas ou aprimoradas nas próximas Sprints.

(Prikladnicki, Willi e Milani, 2014, p. 46).

Nos tópicos a seguir, cada uma dessas cerimônias serão apresentadas.

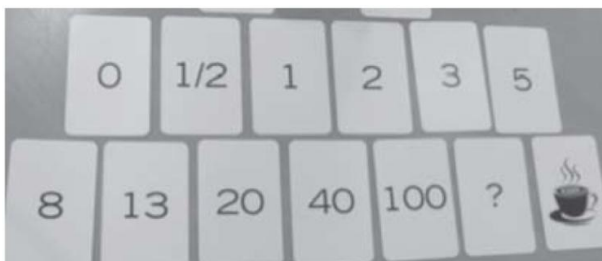
1.9 Planejamento da Sprint

É a primeira reunião do projeto e todos precisam participar, deve ter uma duração máxima de oito horas. O PO e a Equipe de Desenvolvimento irão ordenar os itens do *backlog* por importância das necessidades do negócio e a complexidade para a implementação da funcionalidade,

Antes do início de cada sprint, o product owner enuncia a meta de desenvolvimento para o incremento a ser completado no próximo sprint. O Scrum master e a equipe de desenvolvimento selecionam os itens a serem transferidos para o backlog do sprint. A equipe de desenvolvimento determina o que pode ser entregue no incremento dentro dos limites da janela de tempo alocada ao sprint e, junto ao Scrum master, qual será o trabalho necessário para entregar o incremento. A equipe de desenvolvimento decide quais papéis são necessários e como precisarão ser atendidos. (Pressman e Maxim, 2021, p. 44)

Durante o planejamento da *sprint* pode-se utilizar uma técnica denominada “pôquer do planejamento”, que consiste em uma dinâmica em grupo utilizada para estimar o tamanho do trabalho a ser empregado para realizar. Por fazer uso de cartas, esta técnica dá a impressão a Equipe de desenvolvimento que estão realmente em um jogo, como o pôquer. No entanto, o propósito é lúdico e descontraído para poder realizar uma estimativa que muitas vezes pode ser cansativa e estressante.

No SCRUM não se leva em conta o tempo, ou seja, a duração das tarefas, mas no tamanho delas. Após o time de desenvolvimento entender o objetivo da história, cada membro do time escolhe uma carta, que foi previamente distribuída a todos, e coloca na mesa virada para baixo. As cartas apresentam valores numéricos relacionados com o grau de complexidade da funcionalidade a ser desenvolvida, sob o ponto de vista de cada membro da equipe. O objetivo é obter um consenso relacionado ao tamanho do desenvolvimento da funcionalidade. Ao final, o time calcula a média dos valores apresentados, chegando a uma pontuação final sobre a história apresentada. (Sbrocco e Macedo, 2012, p. 167). Na imagem abaixo é possível verificar um modelo destas cartas que podem ser utilizadas.



Fonte: Sbrocco e Macedo, 2012

1.10 Scrum Diárias

É uma reunião ao mesmo tempo simples e importante, sendo um dos grandes diferenciais do SCRUM. Nessa reunião cada membro do time deve responder sobre o que fez no dia anterior, sobre o que pretende fazer e se há algum impedimento para a conclusão da tarefa sob sua responsabilidade. A reunião diária deve ter uma duração muito rápida, de no máximo quinze minutos, tendo como participantes apenas o time e

o SCRUM Master. (Sbrocco e Macedo, 2012, p. 167)

Quando algum membro identifica a necessidade de obter um detalhamento mais aprofundado ou de oferecer auxílio na remoção de algum impedimento, a questão deve ser tratada após a reunião, envolvendo apenas os participantes necessários. Essa prática contribui para a manutenção da duração planejada da reunião e, consequentemente, para a produtividade do time (Maschietto, Rodrigues e Bianco, 2020, p. 58).

1.11 Revisão da Sprint

A Revisão da *Sprint* consiste em uma reunião de avaliação de tudo o que foi realizado durante o ciclo da Sprint. Para iniciar a reunião, um dos integrantes da equipe de desenvolvimento apresenta a meta da *Sprint*, os itens do *Product Backlog* aos quais o time se comprometeu e todos os itens concluídos integralmente. Em seguida, os membros da equipe demonstram de forma funcional os itens desenvolvidos. Essa reunião deve ter duração máxima de quatro horas e deve contar com a participação do *Product Owner*, do *Scrum Master*, da equipe Scrum e de outros convidados, se necessário. Recomenda-se que a Revisão da *Sprint* seja realizada ao final de cada *Sprint* (Sbrocco e Macedo, 2012, p. 168).

Ao término da apresentação, o *Product Owner* dialoga com os convidados, podendo aceitar ou rejeitar a *Sprint* com base no que foi demonstrado. Caso seja necessária a inserção de novas funcionalidades ou alterações, estas são registradas no *Product Backlog* do produto e priorizadas em momento oportuno.

Na Revisão da Sprint, o Dono do Produto valida ou não as entregas da sprint, de acordo com a meta acordada com a Equipe de Desenvolvimento durante a Reunião de Planejamento da Sprint. A Equipe de Desenvolvimento discute sobre a Sprint, o que correu bem, os problemas enfrentados e as soluções encontradas e, após a demonstração, responde às perguntas dos presentes. Também é nesse momento, quando necessário, que será atualizado qualquer artefato utilizado para determinar o progresso atual do projeto. (Prikladnicki, Willi e Milani, 2014, p. 55)

1.12 Retrospectiva da Sprint

A retrospectiva tem como objetivo identificar os acertos e o que pode ser melhorado em uma *Sprint*. São avaliados aspectos relacionados ao trabalho em equipe, pontos positivos e negativos, e realizadas análises sobre estratégias para melhoria. Devem participar da reunião, obrigatoriamente, o time e o Scrum Master. O *Product Owner* também pode participar, sempre que convidado. Todos os membros da equipe devem responder a duas perguntas: o que foi bom durante a *sprint* e o que se pode fazer para melhorar a próxima. O Scrum Master deve anotar tudo e o time deve priorizar os itens apontados em uma ordem ideal para mudança. A retrospectiva precisa acontecer logo após a revisão da *sprint*, com duração média de três horas. (Sbrocco e Macedo, 2012, p. 168)

Como esta reunião tem por objetivo a melhoria contínua do time e dos processos, os membros da Equipe de Desenvolvimento não devem encarar os eventuais feedbacks como pessoais, mas analisar quando receberem e encarar que há problemas que a equipe precisa se auto-organizar e, com a ajuda do Scrum Master encontrar uma solução. (Prikladnicki; Willi; Milani, 2014, p. 55)

1.13 Artefatos

Na metodologia Scrum, os artefatos são elementos fundamentais que ajudam na transparência, inspeção e adaptação do trabalho da equipe. Eles fornecem informações essenciais para evidenciar o que está sendo feito, o que está em andamento e o que foi concluído. (Maschietto, Rodrigues e Bianco, 2020, p. 60). Neste tópico serão abordados os quatro principais artefatos do Scrum: *Backlog* do produto, *backlog* da *Sprint*, Gráfico *Burndown* e Incremento.

1.14 Backlog do Produto

O *Backlog* do Produto é uma lista de prioridades que foi ordenada e criada pela Equipe Scrum logo no início do projeto, seu objetivo é estabelecer e deixar nítido para todos o que precisa ser entregue ao cliente. Somente o *Product Owner* pode realizar alguma ação de inserção, alteração,

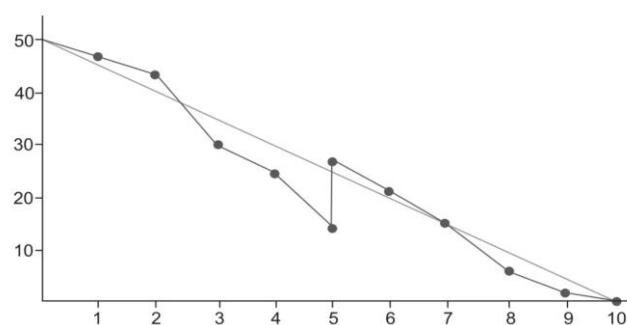
remoção ou modificar as ordens dos itens. (Sbrocco e Macedo, 2012, p. 169)

1.15 Backlog da Sprint

É a relação dos itens selecionados para serem desenvolvidos durante uma *Sprint*. Este *backlog* é criado ao final de cada Reunião de Planejamento da *Sprint*, sendo composto por uma meta, os itens selecionados para o desenvolvimento durante a *Sprint*, e as tarefas técnicas que são necessárias para que este item seja desenvolvido e torne um incremento do produto, ou seja, uma entrega de valor para o cliente. (Prikladnicki, Willi e Milani, 2014, p. 52)

1.16 Gráfico Burndown

Consiste em uma representação em formato de gráfico do andamento da *Sprint*, apresentando quão próximo ou distante a equipe está de atingir a meta. Na imagem apresentada abaixo é possível verificar um gráfico deste tipo, onde na coluna vertical está representada a quantidade de trabalho que ainda precisa ser realizado, e a horizontal, a quantidade de dias da *Sprint*. A linha reta representa o fluxo de trabalho ideal, sendo assim, quando o gráfico está acima desta linha, o time está distante de atingir a meta. Em cima desta linha, o time está no fluxo ideal de trabalho, caso esteja abaixo, o time está entregando de forma antecipada. O Scrum prega que este gráfico precisa ser atualizado diariamente. (Sbrocco; Macedo, 2012, p. 172)



Fonte: Sbrocco e Macedo, 2012

1.17 Incremento

Ao concluir a *Sprint*, o time de desenvolvimento deverá entregar como resultado do que foi produzido no decorrer do período da *Sprint*. O time de desenvolvimento precisa sempre precisar pela qualidade do que está sendo implementado,

pois, estão entregando valor para o cliente, que pode decidir por disponibilizar este incremento em produção. (Maschietto, Rodrigues e Bianco, 2020, p. 62)

2. Metodologia

Este artigo foi desenvolvido com base em uma pesquisa de natureza qualitativa e bibliográfica, cujo objetivo é levantar e analisar conceitos teóricos fundamentais sobre Engenharia de Software e sobre a metodologia ágil de desenvolvimento Scrum. A pesquisa foi conduzida mediante consulta a livros técnicos disponíveis em uma biblioteca virtual, utilizando os seguintes termos de busca: **Engenharia de Software, Metodologias de Desenvolvimento de Software e Scrum.**

O procedimento metodológico adotado segue os critérios de uma pesquisa bibliográfica. Segundo Gil (2008), a pesquisa bibliográfica é realizada com base em material já elaborado, constituído, principalmente, por livros e artigos científicos, sendo especialmente útil para a análise de posicionamentos teóricos e para a formulação de referenciais que sustentem investigações acadêmicas.

A seleção dos materiais considerou critérios de relevância, atualidade e aderência ao tema, priorizando publicações dos últimos dez anos, excetuando-se obras clássicas ou fundamentais para a compreensão dos conceitos-chave. A análise dos dados foi realizada por meio de leitura crítica e interpretativa dos textos selecionados, visando à construção de um referencial teórico capaz de fundamentar as discussões propostas neste trabalho.

3. Conclusão

A transformação dos processos de desenvolvimento de software tem sido impulsionada pela crescente complexidade dos sistemas, pela velocidade da evolução tecnológica e pelas constantes mudanças nas demandas dos usuários. Nesse contexto, os métodos tradicionais, embora historicamente tenham contribuído para a estruturação do desenvolvimento, mostram-se cada vez mais limitados diante da necessidade de adaptabilidade, interatividade e entregas contínuas. Foi nesse cenário que as metodologias

ágeis emergiram como uma resposta inovadora, propondo uma abordagem centrada na colaboração, na flexibilidade e no foco constante no valor entregue ao cliente.

Dentre essas metodologias, o Scrum consolidou-se como uma das práticas mais adotadas e estudadas na engenharia de software contemporânea. Ao longo deste estudo, foi possível analisar sua estrutura conceitual, fundamentada em papéis bem definidos, cerimônias e artefatos. Esses elementos não apenas organizam o trabalho das equipes, mas também promovem transparência, inspeção contínua e adaptação incremental, princípios essenciais para projetos em ambientes incertos e altamente dinâmicos.

A revisão da literatura técnica evidenciou que a adoção do Scrum representa uma alternativa estratégica aos modelos tradicionais, não apenas devido à sua estrutura enxuta e orientada a entregas rápidas, mas também por sua capacidade de alinhar objetivos técnicos aos objetivos de negócio. Ao promover ciclos curtos de desenvolvimento e *feedback* contínuo, a metodologia contribui para aumentar a qualidade do produto, reduzir riscos e criar ambientes de trabalho mais responsivos e inovadores.

Este estudo, ao propor uma análise da adoção do Scrum com base na literatura técnica, oferece subsídios teóricos relevantes para pesquisadores, profissionais e instituições que desejam compreender criticamente os desafios e benefícios dessa metodologia. Para trabalhos futuros, recomenda-se a realização de estudos de caso empíricos que investiguem a aplicação prática do Scrum em diferentes tipos de organizações e contextos de projeto, ampliando o conhecimento sobre sua aplicabilidade no mundo real.

4. Referências

GIL, Antonio Carlos. **Como elaborar projetos de pesquisa**. 5. ed. São Paulo: Atlas, 2008.

HIRAMA, Kechi. **Engenharia de software**. Rio de Janeiro: GEN LTC, 2011. [e-book]. p. 62. ISBN 9788595155404. Disponível em: <https://integrada.minhabiblioteca.com.br/reader/books/9788595155404/> Acesso em: 13 jul. 2025.

MASCHIETTO, Luís G.; RODRIGUES, Thiago N.; BIANCO, Clécides M. D.; et al. **Processos de desenvolvimento de software**. Porto Alegre: SAGAH, 2020. [e-book]. p. 34. ISBN 9786556900520. Disponível em: <https://integrada.minhabiblioteca.com.br/reader/books/9786556900520/> Acesso em: 13 jul. 2025.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**. 9. ed. Porto Alegre: AMGH, 2021. [e-book]. p. 8. ISBN 9786558040118. Disponível em: <https://integrada.minhabiblioteca.com.br/reader/books/9786558040118/> Acesso em: 13 jul. 2025.

PRIKLADNICKI, Rafael; WILLI, Renato; MILANI, Fabiano. **Métodos ágeis para desenvolvimento de software**. Porto Alegre: Bookman, 2014. [e-book]. p. 46. ISBN 9788582602089. Disponível em: <https://integrada.minhabiblioteca.com.br/reader/books/9788582602089/> Acesso em: 13 jul. 2025.

RUBIN, Kenneth S. **Scrum essencial: um guia prático para o mais popular processo ágil**. Rio de Janeiro: Editora Alta Books, 2017. [e-book]. p. 13. ISBN 9788550804118. Disponível em: <https://integrada.minhabiblioteca.com.br/reader/books/9788550804118/> Acesso em: 13 jul. 2025.

SBROCCO, José Henrique Teixeira de C.; MACEDO, Paulo Cesar de. **Metodologias ágeis – engenharia de software sob medida**. Rio de Janeiro: Érica, 2012. [e-book]. p. 36. ISBN 9788536519418. Disponível em: <https://integrada.minhabiblioteca.com.br/reader/books/9788536519418/> Acesso em: 13 jul. 2025.